Rack to the Future — Technical Spec

Oct 18, 2026 · @Jason - MedVR

Summary

Rack to the Future is an Apple Vision Pro app that lets a team design and inspect AI data centers as one multi-scale digital twin, from a single chip out to a fleet of sites on Earth, in orbit, on the Moon and on Mars. It is built for the SundAI MIT AI Hackathon challenge to design the 2035 data center, but works on today's facilities too.

Problem. Data center decisions about chips, cooling, power, buildings, cost and public reaction live in separate spreadsheets and tools. Nobody sees how a choice at one scale (a 5.5 kW chip) forces choices at another (liquid cooling, grid capacity, siting).

What the app does. You zoom continuously through the scales, switch analytical lenses at any scale, and move a time slider from 2025 to 2035. Constraints such as budget, grid cap and launch cost are adjustable presets, not hard rules, so the trade-offs play out live.

Goals for the prototype

- A working zoom from rack to fleet with at least two lenses (heat and power) live at every level.
- A scenario panel where the challenge numbers are default presets the user can change.
- Constraint checks that visibly flag the first limit a scenario hits.
- A 3–5 minute demo that tells one clear story.

Non-goals for the hackathon

- Engineering-grade CFD or electrical simulation; we use simplified models.
- Live integration with real facility telemetry.
- A complete optimizer that finds the best fleet; the app shows consequences, people make the choices.

Users and use cases

The primary user is a planning team (infrastructure, finance, comms) sitting together around one shared model. The app should answer "what happens if we change this?" in seconds.

User	Question they bring	What they use
Infrastructure architect	Which racks, cooling and sites fit our power and budget?	Scale ladder, heat and power lenses, constraint checks
Finance / business lead	What does the fleet cost over 5 years, and where does money run out?	Business case lens, budget gauge, scenario presets
Comms / community lead	Where will a site meet local resistance, and why?	Comms lens with sentiment at the site level
Manufacturing / supply chain	What has to be built, and in what order?	Manufacturing lens, building and rack scales
Hackathon judges	Does this design serve the chosen user case at the constraints?	Demo flow, fleet view, constraint panel

Core use cases:

1. Build a fleet scenario from presets (user case A, B or C; rack mix; sites) and see which constraint binds first.
2. Zoom from a fleet-level warning down to its cause (for example, a grid overload traced to one hall's rack mix).
3. Compare 2025, 2030 and 2035 versions of the same facility on the time slider.
4. Walk through a single hall in immersive mode to review layout, heat and cooling with the team.

Scale ladder

The twin has seven zoom levels, and every level uses the same data model, so a number changed at one scale flows up and down the ladder. Each level picks the visionOS presentation that suits its physical size.

Level	What you see	visionOS presentation	Key quantities
1. Chip	One GPU, LPU or air card package	Volume, exploded view	Power per chip (kW), heat flux, junction temperature
2. Rack	GPU, LPU, air or GB200 NVL72 rack	Volume on the table	Rack kW, chip count, cost, cooling type
3. Hall / pod	Rows of racks, cooling loops, power distribution	Full immersive space you walk through	Hall MW, hot and cold aisles, cooling capacity
4. Building	Halls, power plant, chillers or towers	Volume, cutaway	Facility MW, PUE, water use, build cost
5. Site	Building plus grid tie, land and surroundings	Volume, terrain map	Grid allocation, climate, energy price, carbon
6. Fleet (Earth)	All Earth sites on a globe	Volume, globe	Total MW against the 500 MW cap, latency to users
7. Off-world	Low Earth orbit, Moon, Mars	Volume, orbital view	Launch mass and cost, radiator area, round-trip latency

Navigation: a pinch on any object zooms one level down into it, and a back gesture or breadcrumb bar zooms out. A persistent window beside the volume shows the scenario panel and gauges at every level.

Lenses

A lens is an analytical overlay that renders at every scale. Switching lens changes what the colors, labels and gauges mean without moving the camera.

Lens	At chip / rack	At hall / building	At site / fleet	Prototype priority
Heat (architecture)	Heat flux per chip; cooling type required	Airflow, liquid loops, hot spots	Heat rejected, water draw, climate fit	Must have

Lens	At chip / rack	At hall / building	At site / fleet	Prototype priority
Power signature	kW per chip and rack	Load curve over a day; prefill vs decode	MW per site against grid cap; daily demand shape by user case	Must have
Business case	Cost per rack and per kW	Build cost, PUE-driven energy cost	5-year spend against the $30B budget	Must have
Manufacturing	Chip and rack supply, lead time	Prefab modules, build sequence	Build-out order across sites	Nice to have
Comms (sentiment)	None	None	Pro/con sentiment from social media per site	Nice to have
Physics simulation	Thermal model of a chip or cold plate	Simplified airflow and coolant flow	Orbital radiator sizing	Stretch

The business case and comms lenses are linked: negative sentiment at a site raises its risk score and can add delay or cost to its build-out in the business case.

A photoreal mode ("Reality HD") can sit beside the schematic mode for halls and buildings. Schematic is the default because it keeps lens colors readable.

Scenario model

Every number from the challenge slides is a default preset, not a fixed requirement. A scenario is a set of parameter values; the engine derives metrics from it and runs constraint checks after every change.

Parameters

Parameter	Default (challenge preset)	Range in the app
5-year budget (chips, buildings, energy, launches)	$30B	$5B to $100B

Parameter	Default (challenge preset)	Range in the app
Earth grid power, all sites together	500 MW	100 MW to 2 GW
Launch cost to orbit	$200/kg	$50 to $2,000/kg
User case	A, B or C, or a hybrid mix	Mix weights summing to 100%
Earth sites	6	1 to 12
Off-world locations	LEO, Moon, Mars	On or off per location
Rack catalog	GPU, LPU, air, GB200 NVL72 (see Appendix)	Editable per rack type
Cooling per hall	Air + chillers, air + evaporative, direct liquid, immersion, radiators in orbit	One per hall
PUE	1.2 (assumption)	1.05 to 1.6
Energy price	$0.05/kWh (assumption)	$0.02 to $0.20/kWh

Derived metrics

```
P_{facility} = PUE \times \sum_{racks} P_{rack}
```

```
Racks_{max} = \frac{P_{grid}}{PUE \times P_{rack}}
```

```
Cost_{5yr} = Cost_{racks} + Cost_{buildings} + P_{facility} \times
43{,}800\,h \times price + Cost_{launch}
```

Per-user demand comes from the user case: total peak tokens per second = target users × peak tokens/s per user, split into output (decode) and input (prefill).

Constraint checks

The app runs these after every change and shows the first one that fails as a red gauge, with a tap that zooms to its cause:

- **Grid:** Earth facility power must stay at or under the grid cap.

- **Budget:** 5-year cost must stay at or under the budget.
- **Serving:** delivered prefill and decode throughput must cover 100% of the chosen case at peak.
- **Cooling fit:** each rack's power per chip must suit its hall's cooling type (for example, a 792 kW GPU rack cannot be air cooled).
- **Latency:** interactive work must run where round-trip time allows; Moon and Mars only take background or batch work.

Worked example with the defaults

With 500 MW and no cooling overhead, the grid holds about 631 of the 2035 GPU racks (792 kW each), which is about $15.0B of hardware. Hardware plus buildings and roughly $1.1B of energy over 5 years at $0.05/kWh still fits under $30B. So with these presets the grid binds before the budget does, and further growth has to come from off-world capacity or from more tokens per watt (for example, LPU racks for decode). This is the core story the demo shows.

Time horizon

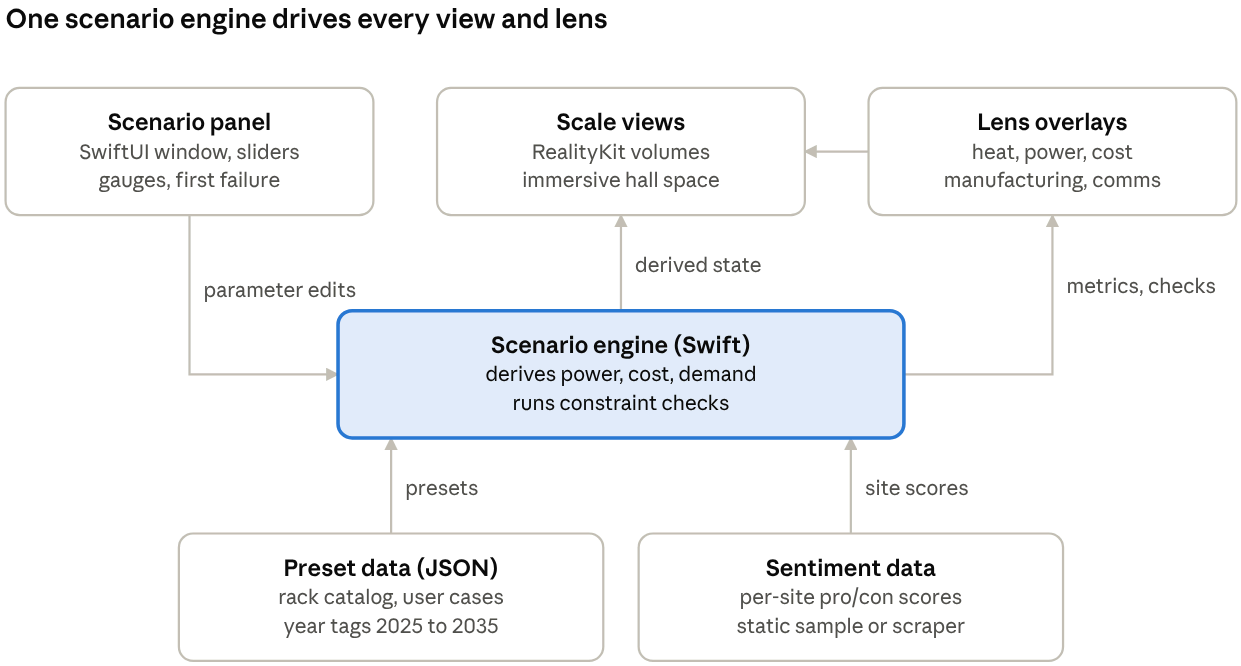
A time slider runs from 2025 to 2035, with 2030 projection data as the midpoint. The same facility morphs as the slider moves: rack types swap, power density rises, and cooling options change.

- **2025:** GB200 NVL72 racks (132 kW, 72 GPUs, about $3–5M) as the baseline. This makes the app immediately useful on real facilities with today's data.
- **2030:** projection data drives the middle of the range; values between known points are interpolated.
- **2035:** the challenge's GPU, LPU and air racks.

Each rack type and parameter carries a year tag, so a scenario can be replayed at any year. Moving the slider re-runs the constraint checks, which shows when a design that fits in 2025 stops fitting.

App architecture

The app is built around one pure-Swift scenario engine; every view and lens reads from it, and only the scenario panel writes to it.



app architecture · engine, views, lenses, data

A parameter edit in the panel re-runs the engine, and the views and lenses redraw from its derived state.

visionOS scenes

- **Scenario panel:** a SwiftUI `WindowGroup` with sliders, the year slider, gauges and the first failing constraint. It stays open at every scale.
- **Volumes:** volumetric windows for the chip, rack, building, site, fleet and off-world levels, each a RealityKit scene.
- **Immersive space:** an `ImmersiveSpace` for the hall level, so the team can walk the rows.

Modules

Module	Responsibility	Tech
ScenarioEngine	Data model, derived metrics, constraint checks; no UI code	Swift package, `@Observable` state, unit tests
ScaleViews	Procedural models for each level, zoom navigation, breadcrumb	RealityKit, SwiftUI

Module	Responsibility	Tech
Lenses	Map engine metrics to colors, labels and gauges at each level	RealityKit materials, SwiftUI overlays
PresetData	Challenge presets, rack catalog, year tags	Bundled JSON
Sentiment	Per-site pro/con scores for the comms lens	Static JSON for the demo; scraper as stretch

Data model

Scenario holds the parameters and a list of Sites. A Site has a location (Earth, LEO, Moon or Mars), a grid allocation and Buildings. A Building has Halls; a Hall has a cooling type and a rack mix. A RackType carries power, chip count, cost and year, and points to a ChipType. UserCase holds target users, reads per user per day and peak tokens per second.

Models are generated from simple shapes rather than detailed assets, so the scale views stay fast and the hackathon build stays small.

Demo flow

The demo tells one story: the grid runs out before the money does, and the twin shows where compute has to go next.

1. Open on the fleet globe with the challenge presets loaded and user case B (agent developers) selected.
2. Pinch into one Earth site, then a building, then a hall. Walk the hall in immersive mode with the heat lens on.
3. Pinch into a GPU rack and a chip: 5.5 kW per chip, flagged as needing direct liquid or immersion cooling. Switch the hall to air cooling to show the red warning, then switch back.
4. Zoom back out to the fleet and add GPU racks. The grid gauge hits 500 MW while the budget gauge still has room.
5. Pull the camera back to low Earth orbit. Turn on LEO capacity and watch the launch cost appear in the business case lens.
6. Drag the time slider back to 2025 to show the same tool on today's GB200 facilities.
7. Close on the title: Rack to the Future.

Build plan

The plan has four parallel workstreams, built in priority order so the demo works even if stretch items slip. Owners are open; assign one person per workstream.

Workstream 1: Scenario engine (pure Swift, no UI)

- [ ] Data model: scenario, site, building, hall, rack type, chip type, user case, all with year tags
- [ ] Load the challenge presets from a JSON file
- [ ] Derived metrics: facility power, rack count, 5-year cost, peak token demand
- [ ] Constraint checks with a "first failing constraint" result
- [ ] Unit tests against the worked example (about 631 GPU racks under 500 MW)

Workstream 2: Scales and navigation (RealityKit)

- [ ] Procedural rack, hall and building models from simple shapes
- [ ] Rack and fleet volumes; hall immersive space
- [ ] Pinch to zoom in, breadcrumb to zoom out
- [ ] Stretch: chip exploded view, off-world orbital view

Workstream 3: Lenses and gauges (SwiftUI + RealityKit materials)

- [ ] Heat lens: color by kW per chip and cooling fit
- [ ] Power lens: MW per site against the grid cap
- [ ] Business case gauge: spend against budget
- [ ] Scenario panel window with sliders bound to the engine
- [ ] Stretch: manufacturing and comms lenses

Workstream 4: Data, demo and pitch

- [ ] Rack catalog and 2025 / 2030 / 2035 values in JSON
- [ ] Sample sentiment data for 2–3 sites (static, if scraping is out of time)
- [ ] Demo script rehearsed against the Demo flow above
- [ ] Pitch: title and short elevator pitch

Integration order: engine first, then the fleet and rack volumes wired to it, then the heat and power lenses, then the hall immersive space. Stretch items start only once the demo flow runs end to end.

Open questions and risks

- [ ] Is the comms lens scraping live social media, or using a static sample for the demo? Live scraping adds API and terms-of-service risk.
- [ ] Which user case does the demo commit to: A, B, C or a hybrid?
- [ ] What chip throughput (tokens/s per chip) do we assume? The slides give demand but not supply, so the serving check needs a stated assumption.
- [ ] Building cost per MW is not on the slides; we need one assumed value for the budget check.
- [ ] What does "Reality HD" cover: photoreal halls only, or also captured scans of real sites?

Risks

- **Device time:** only Vision Pro hardware shows the immersive hall properly. Mitigation: build and test most scales in the simulator, and book device time for the hall.
- **Scope:** seven scales and six lenses is a lot for a hackathon. Mitigation: the build plan's priority order; rack and fleet scales with heat and power lenses are enough for the demo.
- **False precision:** simplified models can look more exact than they are. Mitigation: label assumptions in the scenario panel and round displayed values.

Appendix: reference numbers

These come from the SundAI MIT AI Hackathon challenge slides and are the app's default presets. Per-chip power, cost per kW and peak totals are derived here.

Rack catalog

Rack	Year	Power (kW)	Chips	Cost	kW per chip	Cost per kW	Max racks at 500 MW (no overhead)
GPU rack	2035	792	144	$23.8M	5.5	~$30k	~631
LPU rack	2035	516	256	$14.3M	2.0	~$28k	~969

Rack	Year	Power (kW)	Chips	Cost	kW per chip	Cost per kW	Max racks at 500 MW (no overhead)
Air rack	2035	37	32 cards	$0.5M	1.2	~$14k	~13,500
GB200 NVL72	2025	132	72 GPUs	~$3–5M	1.8	~$23–38k	~3,790

Chip roles from the slides: GPU with 3 TB memory for prefill, LPU for fast decode, air card as the cheap option.

User cases

Case	Daily usage	Target users	Reads per user per day (tokens)	Peak tokens/s per user	Peak total output (tokens/s)	Peak total input (tokens/s)
A: Prompters	30 chat prompts	4,000,000,000	1.9M	2.4 out + 5.6 in	9.6B	22.4B
B: Agent developers	12 interactive + 30 background agent tasks	50,000,000	860M	28 out + 333 in	1.4B	16.7B
C: Knowledge workers	20 prompts + 1 interactive + 4 background tasks	400,000,000	104M	4.6 out + 40 in	1.8B	16.0B

One agent developer reads as many tokens as about 450 prompters. Case A needs the most decode throughput; prefill demand is similar across all three.

Scoring from the slides: users served at the constraints first, then dollars saved.